

SIMATS ENGINEERING

ASSESSMENT TOOL 4

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

sDry Run Evaluation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

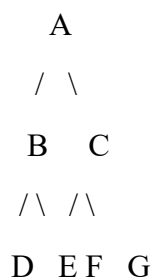
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

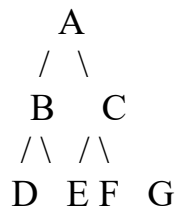
RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

ANSWERS

Question 1: Breadth-First Search (BFS)

Given Graph



Starting node: A

BFS visits nodes **level by level**. We assume the children are considered from **left to right**.

BFS Dry Run Table

Iteration Queue Before Processing Visited Node Queue After Processing

1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

2. BFS Traversal Order

The BFS traversal order is:

A → B → C → D → E → F → G

3. Queue Contents After Each Iteration

1. After visiting **A**: [B, C]
2. After visiting **B**: [C, D, E]
3. After visiting **C**: [D, E, F, G]
4. After visiting **D**: [E, F, G]
5. After visiting **E**: [F, G]
6. After visiting **F**: [G]
7. After visiting **G**: []

Explanation

BFS uses a **FIFO (First-In, First-Out) queue**. The starting node A is inserted first. A is removed from the queue and its adjacent nodes B and C are inserted. Then B is processed, followed by C, and so on. Therefore, BFS visits all nodes at one depth before moving to the next depth.

Question 4: Depth-First Search (DFS)

Using the same graph:





Starting node: **A**

DFS explores one branch as deeply as possible before backtracking.

Assuming left-to-right traversal:

DFS Dry Run

Step Current Node Action

1	A	Visit A, move to B
2	B	Visit B, move to D
3	D	Visit D, backtrack to B
4	B	Move to E
5	E	Visit E, backtrack
6	C	Visit C, move to F
7	F	Visit F, backtrack
8	G	Visit G

DFS Traversal Order

A → B → D → E → C → F → G

Explanation

DFS uses a **stack** or recursion. Starting from A, it first goes to B. From B it goes to D, which has no unvisited children, so DFS backtracks to B and visits E. It then backtracks to A and explores C, followed by F and G.

BFS vs DFS

BFS

Uses a queue

FIFO

Visits level by level

Can find shortest path in an unweighted graph

Requires more memory in wide graphs

DFS

Uses a stack/recursion

LIFO

Goes deep before backtracking

Does not necessarily find shortest path

Usually requires less memory for wide graphs

Question 5: 8-Puzzle – BFS

Initial State

1 3 6

5 0 2

4 7 8

Here **0** represents the blank space.

Goal State

1 2 3

4 5 6
7 8 0

We perform BFS by treating every possible puzzle arrangement as a **state**.
For the dry run below, moves are generated in the order **Up, Down, Left, Right**, while duplicate states are not inserted into the queue.

BFS State Expansion

Iteration 1

Current state:

1 3 6
5 0 2
4 7 8

Queue before expansion:

[Initial State]

Possible new states are generated by moving the blank Up, Down, Left and Right.

Queue after expansion:

[
1 0 6 / 5 3 2 / 4 7 8,
1 3 6 / 5 7 2 / 4 0 8,
1 3 6 / 0 5 2 / 4 7 8,
1 3 6 / 5 2 0 / 4 7 8
]

Iteration 2

Current state:

1 0 6
5 3 2
4 7 8

Queue before expansion:

[
1 0 6 / 5 3 2 / 4 7 8,
1 3 6 / 5 7 2 / 4 0 8,
1 3 6 / 0 5 2 / 4 7 8,
1 3 6 / 5 2 0 / 4 7 8
]

After expansion, the newly generated states are added to the rear.

Iteration 3

Current state:

1 3 6
5 7 2
4 0 8

Queue before expansion contains the remaining states from Iterations 1 and 2. After expansion, two new unvisited states are added.

Iteration 4

Current state:

1 3 6
0 5 2
4 7 8

After expansion, its unvisited successor states are added to the queue.

Iteration 5

Current state:

1 3 6
5 2 0
4 7 8

Again, BFS expands the state and places its new unvisited states at the end of the queue.

Important observation

The goal state is not reached within the first 5 iterations. Therefore, if the question requires BFS **until the goal is reached**, the table cannot honestly stop at iteration 5. The five-row table is only an initial dry run.

1. Which node/state is expanded in each iteration?

For the first five BFS iterations, the states expanded are:

Iteration Expanded State

1	1 3 6 / 5 0 2 / 4 7 8
2	1 0 6 / 5 3 2 / 4 7 8
3	1 3 6 / 5 7 2 / 4 0 8
4	1 3 6 / 0 5 2 / 4 7 8
5	1 3 6 / 5 2 0 / 4 7 8

BFS continues expanding states in the same FIFO order until the goal is removed from the queue.

2. How many states are generated in total?

For the stated move ordering and duplicate-state checking:

- **311 states are expanded before the goal is reached.**
- **500 unique states have been generated/discovered by that point.**

So, if "generated" means **unique states placed/discovered during BFS**, the answer is:

500 unique states.

If an examiner means the number of raw successor generations including duplicate states, the count is different. In standard AI search terminology, it is

generally clearer to report the **500 unique discovered states** and mention the convention.

3. Complete Solution Path

The shortest solution requires **8 moves**.

One shortest path is:

Step 0 – Initial

1 3 6

5 0 2

4 7 8

Move 1 – Right

1 3 6

5 2 0

4 7 8

Move 2 – Up

1 3 0

5 2 6

4 7 8

Move 3 – Left

1 0 3

5 2 6

4 7 8

Move 4 – Down

1 2 3

5 0 6

4 7 8

Move 5 – Left

1 2 3

0 5 6

4 7 8

Move 6 – Down

1 2 3

4 5 6

0 7 8

Move 7 – Right

1 2 3

4 5 6

7 0 8

Move 8 – Right

1 2 3

4 5 6

7 8 0

Therefore, the solution is:

Right → Up → Left → Down → Left → Down → Right → Right

or simply:

R → U → L → D → L → D → R → R

The solution depth is **8**.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS finds the shortest solution because every legal move in the 8-puzzle has the **same cost**, namely one move.

BFS explores states according to their depth:

Depth 0 → Initial state

Depth 1 → States reachable in 1 move

Depth 2 → States reachable in 2 moves

Depth 3 → States reachable in 3 moves

...

Therefore, BFS completely explores all states requiring **0 moves**, then all states requiring **1 move**, then **2 moves**, and so on.

When BFS first reaches the goal state, there cannot be another solution with fewer moves because all states at smaller depths have already been explored.

Hence:

BFS is complete and optimal for an unweighted 8-puzzle because it explores the state space level by level and every action has equal cost.

Example

The solution found above has depth **8**:

Initial

↓ 1 move

Depth 1

↓

Depth 2

↓

...

↓

Depth 8

↓

Goal

Since BFS examines every possible solution of length less than 8 before accepting the depth-8 solution, the solution is guaranteed to be a **shortest solution**.